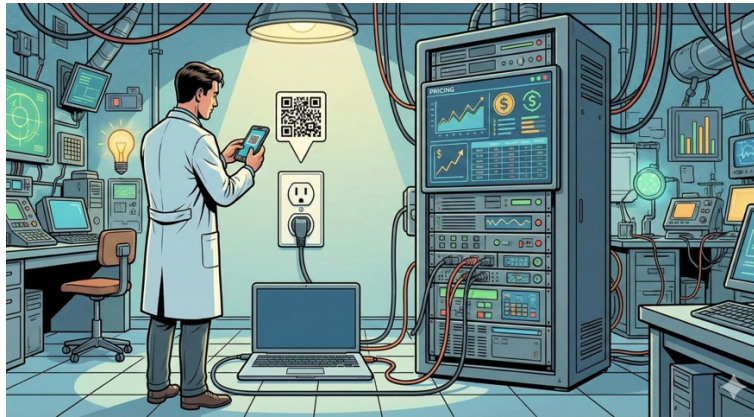


CAHIER DES CHARGES



1. Contexte

L'établissement scolaire souhaite mettre en place un système de gestion de recharge pour appareils mobiles. Ce service permettra aux élèves de recharger leurs appareils grâce à des prises électriques dédiées, tout en contrôlant la consommation et la facturation par kilowattheure.

2. Objectifs

1. Fournir un système de recharge électrique fonctionnel et automatisé pour les élèves.
2. Facturer les élèves en fonction de leur consommation électrique.
3. Offrir un crédit initial en début d'année scolaire.
4. Permettre aux élèves de recharger leur crédit via PayPal en cas de dépassement.
5. Gérer les prises électriques via un dashboard.
6. Fournir une interface utilisateur intuitive pour accéder aux fonctionnalités du système.

3. Fonctionnalités attendues

3.1. Système API Node.js

- Vérification du crédit disponible pour chaque élève connecté.
- Autorisation ou refus de la recharge en fonction du crédit.
- Gestion des recharges du crédit via PayPal.
- Stockage des données utilisateurs (crédit, historique de consommation) et prises électriques dans une base de données.

3.2. Dashboards

3.2.1. Dashboard utilisateur (élèves)

- Interface pour recharger le mobile.
- Affichage du crédit restant et de l'historique de consommation.
- Fonctionnalité de paiement via PayPal pour recharger le crédit.

3.2.2. Dashboard administrateur (gestionnaire réseau)

- Visualisation et gestion de l'état des prises (occupée, libre, hors service).
- Suivi de la consommation globale et par utilisateur.
- Gestion des utilisateurs (ajout, suppression, crédit initial).
- Supervision et maintenance des prises électriques.

3.3. Application mobile

- Identification de l'utilisateur via un compte personnel.
- Recharge du crédit avec intégration PayPal.
- Identification des prises électriques grâce au scan de QR codes uniques.
- Historique des consommations et paiements.

4. Architecture technique

4.1. Infrastructure

- Hébergement : Mise en production sur une machine virtuelle (VM) DigitalOcean.
- Conteneurisation Docker :
 - Un conteneur dédié pour chaque dashboard (utilisateur et administrateur).
 - Un conteneur pour l'API développée en Node.js.
 - Un conteneur pour la base de données (MySQL, MongoDB, ou autre au choix).
- Reverse Proxy : Utilisation d'un serveur web NGINX pour gérer le routage des requêtes vers les différents conteneurs.

4.2. Communication et messagerie

- Intégration d'un broker MQTT hébergé sur HiveMQ pour gérer la communication entre l'application et les prises connectées.

5. Contraintes techniques

5.1 API et Backend :

- Développement en Node.js.
- Connexion à une base de données (MySQL, MongoDB ou autre au choix).
- Sécurisation des transactions PayPal.
- Respect des normes RGPD pour les données personnelles.

5.2 Dashboards :

- Développement web responsive.
- Compatible avec les navigateurs modernes (Chrome, Firefox, Edge).

5.3 Application mobile :

- Développement en React Native ou une technologie multiplateforme équivalente.
- Support des plateformes Android et iOS.
- Intégration d'un scanner QR code.

5.4 Conteneurisation Docker :

- Docker sera utilisé pour isoler les services (API, dashboards, base de données).
- Configuration des conteneurs avec des fichiers Dockerfile et gestion via docker-compose ou autre outil équivalent.

5.5 Reverse Proxy :

- Configuration d'NGINX pour gérer les connexions et assurer la redirection vers les services correspondants.

5.6 Sécurité :

- Authentification sécurisée des utilisateurs (JWT ou OAuth).
- Protection contre les attaques courantes (injection SQL, XSS, CSRF).
- Certificats SSL/TLS pour sécuriser les connexions.

6. Livrables

6.1 API Node.js fonctionnelle.

6.2 Base de données opérationnelle avec les données des utilisateurs et des prises.

6.3 Dashboard utilisateur.

6.4 Dashboard administrateur.

6.5 Application mobile disponible sur les stores (Android et iOS).

Location Prise de recharge

- 6.6 Fichiers Dockerfile et configurations nécessaires au déploiement.
- 6.7 Documentation technique et utilisateur.

Synoptique du projet :

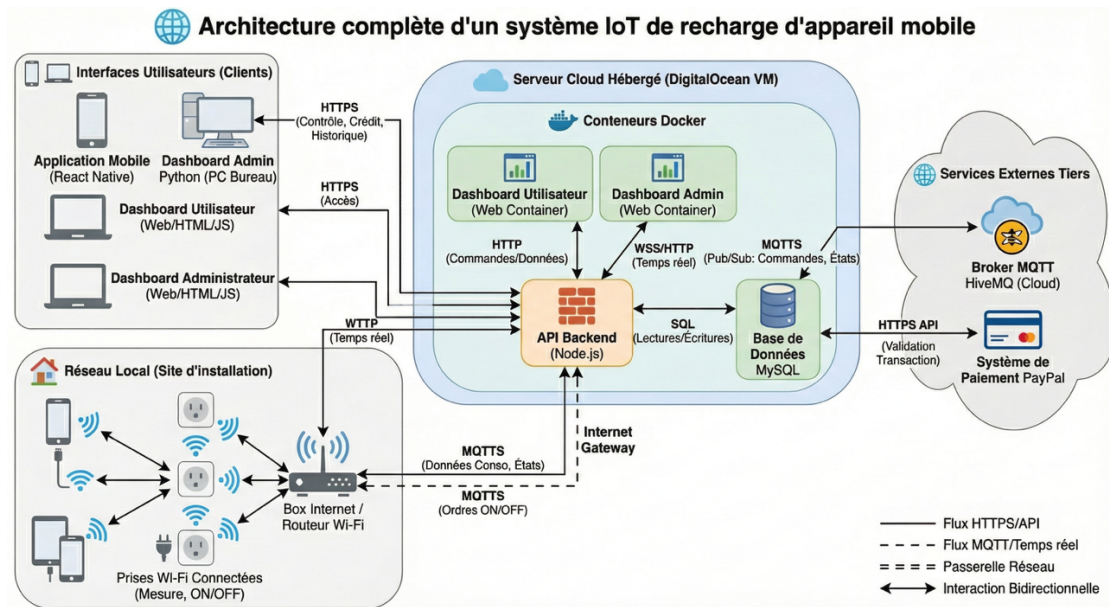
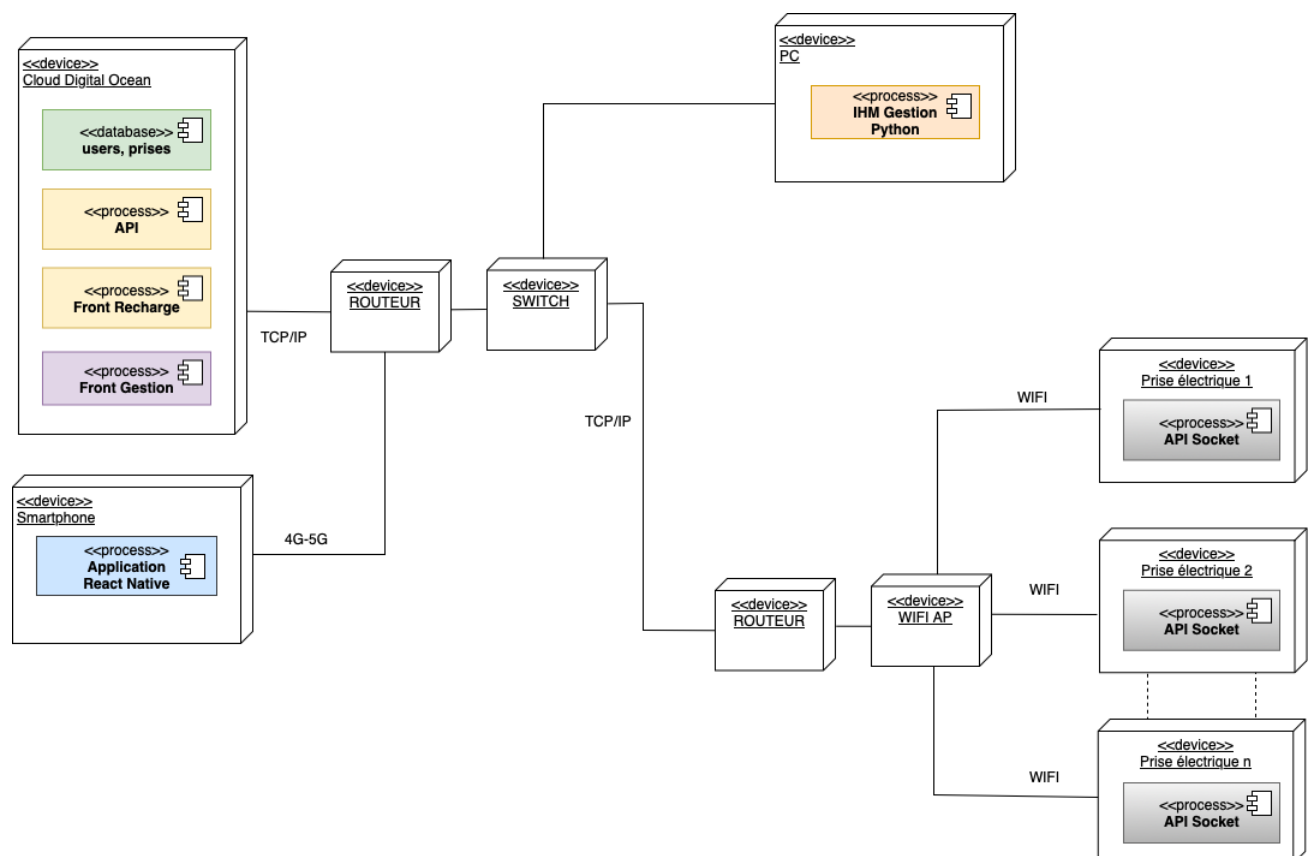


Diagramme de déploiement :



Location Prise de recharge

Nom étudiant	Fonctions à développer	Tâches à effectuer	Matériels et logiciels nécessaires
Yanis Belramoul	Coder l'application de gestion des prises	- Installation des prises wifi. - Installation de l'AP wifi et du routeur du réseau. - Codage de l'API. - Codage du dashboard de recharge - Création du broker mqtt sur hivemq. - Déploiement des conteneurs API, Front et base de données en production. - Schémas SysML (diagrammes de cas d'utilisation et diagrammes de séquence)	- Prise Shelly - NodeJs - Paypal pour developpeur - PC desktop -
Mehdi Takkiedine	Coder l'application smartphone	- Installation des prises wifi. - Installation de l'AP wifi et du routeur du réseau. - Codage de l'application smartphone en React Native. - Création du broker mqtt sur hivemq. - Schémas SysML (diagrammes de cas d'utilisation et diagrammes de séquence)	- PC desktop - React native
Youssef Yajjou	Coder une interface de gestion du réseau des prises en javascript.	- Installation des prises wifi. - Installation de l'AP wifi et du routeur du réseau. - Codage du dashboard de gestion du réseau wifi en html-css-javascript. - Création du broker mqtt sur hivemq. - Déploiement des conteneurs Front. - Schémas SysML (diagrammes de cas d'utilisation et diagrammes de séquence, diagramme de classe)	- PC Desktop
Vijayaraj BARTASSARADYNAIDOU	Coder une interface de gestion du réseau des prises en python	- Installation des prises wifi. - Installation de l'AP wifi et du routeur du réseau. - Codage du dashboard de gestion du réseau wifi en python. - Création du broker mqtt sur hivemq. - Déploiement des conteneurs Front. - Schémas SysML (diagrammes de cas d'utilisation et diagrammes de séquence, diagramme de classe)	- PC Desktop

Lycée Newton : Système de Recharge Connecté

FONCTIONNALITÉS POUR LES UTILISATEURS



Recharge simplifiée via une application mobile.

Les élèves scannent un QR code sur la prise pour démarrer la recharge.



Gestion du crédit et paiement intégré.

Les élèves reçoivent un crédit initial et peuvent le recharger via PayPal.



Supervision complète pour les administrateurs.

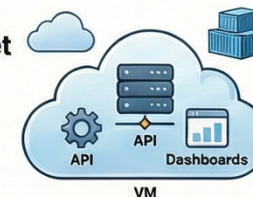
Un dashboard permet de gérer l'état des prises, les utilisateurs et la consommation.



ARCHITECTURE TECHNIQUE DU SYSTÈME IoT

Infrastructure Cloud et conteneurisation.

L'API et les dashboards sont hébergés sur une VM DigitalOcean avec Docker.



Communication en temps réel avec les prises.

Un broker MQTT (HiveMQ) gère les commandes (ON/OFF) et les données des prises.



Technologies modernes et multiplateformes.

L'API est en Node.js et l'application mobile en React Native (iOS/Android).

